

DevMap V2

■ Powered by Groq AI

Complete Implementation Plan

Explain topics from your code · Quiz mode · Smart next-topic suggestions
Live in both VS Code sidebar and the web dashboard

Stack: TypeScript · VS Code API · Groq SDK · llama3-70b-8192
Builds on top of: DevMap V1 (AST scanner + TreeView + WebviewPanel)

What Changes in V2?

V1 already scans your code and shows which topics you have covered. V2 adds a Groq AI layer on top — so DevMap doesn't just track what you used, it explains it in context of your own code, quizzes you on it, and tells you what to learn next. The AI responses appear both in VS Code (hover + sidebar) and in the web dashboard.

Feature	V1	V2 (Groq added)
Topic detection	✓ Done	✓ Same — unchanged

Sidebar TreeView	✓ Done	✓ + Explain button per topic
Web dashboard	✓ Done	✓ + AI panel (explain/quiz/suggest)
Topic explanation	—	✓ Groq explains from YOUR code
Quiz mode	—	✓ Groq generates Q&A per topic
Next topic suggestions	—	✓ Groq picks what to learn next
Code snippet context	—	✓ Actual lines sent to Groq
Groq API key management	—	✓ VS Code SecretStorage
Streaming responses	—	✓ Typewriter effect in dashboard

Why Groq? Groq runs llama3-70b-8192 at ~500 tokens/sec — fast enough for real-time streaming inside VS Code without the user feeling a lag. The free tier (6,000 requests/day) is more than enough for a dev tool.

New Architecture — What Gets Added

V1 data flow: File watcher → AST scanner → State store → Sidebar / Dashboard. V2 adds a Groq service layer that sits between the state store and the UI. When you click Explain, Quiz, or Suggest, the extension pulls your actual code snippet from the state store, builds a prompt, calls Groq, and streams the response back to both the sidebar webview and the dashboard panel.

New file	Role
src/groqService.ts	Wraps Groq SDK — one function each for explain(), quiz(), suggest(). Handles streaming.
src/promptBuilder.ts	Builds prompts from topic name + actual code snippet extracted by AST scanner.
src/secretStore.ts	Stores the Groq API key securely using VS Code's context.secrets API.
src/aiPanel.ts	New sidebar WebviewPanel — renders streaming AI responses with typewriter effect.
data/prompts.json	Prompt templates for each mode (explain / quiz / suggest) — easy to tweak.

Phase 1

Groq SDK Setup + API Key Management

~1 hour · foundation for everything

Goal

Install the Groq SDK, get your API key stored securely inside VS Code (never in plaintext), and verify you can make a basic completion call. Don't skip secure storage — hardcoding keys in settings.json is a bad habit.

Steps

Install SDK	npm install groq-sdk — official Node.js SDK, works in the VS Code extension process directly.
SecretStorage	Use context.secrets.store('devmap.groqKey', key) and context.secrets.get('devmap.groqKey') — built into VS Code, encrypted at rest.
Register command	Add command devmap.setApiKey that shows vscode.window.showInputBox({ password: true }) and saves to SecretStorage.
Create groqService	src/groqService.ts exports a GroqService class. Constructor calls context.secrets.get() to load the key and init the Groq client.
Test call	In groqService, add a ping() method: call groq.chat.completions.create with model: 'llama-3.3-70b-versatile' and a hello message. Log the response.
Error handling	If key is missing, show vscode.window.showWarningMessage('Set your Groq key: Cmd+Shift+P → DevMap: Set API Key').

Key snippet — initialising the Groq client:

```
import Groq from 'groq-sdk';
const key = await context.secrets.get('devmap.groqKey');
const groq = new Groq({ apiKey: key });
```

Phase 2

Prompt Builder — Context-Aware Prompts

~2 hours · the quality of AI = quality of prompts

Goal

Build a prompt factory that takes a topic name + the actual code snippet where you used it, and produces a tight, focused prompt for each of the three modes. The code snippet is what makes Groq's answer feel personal — it explains YOUR code, not a generic example.

The 3 prompt modes:

Mode	Trigger	What gets sent to Groq	Expected output
Explain	Click topic in sidebar	Topic name + code snippet (max 30 lines) where the topic was detected	2–3 para explanation tied to the user's actual code

Quiz	Quiz button in dashboard	Topic name + covered status + user's code context	3 MCQ or short-answer questions + answers
Suggest	Auto after each scan	Full list of covered topics + uncovered topics	Top 3 topics to learn next with 1-line reason each

Example — Explain prompt template:

```
You are a senior JS/Node.js tutor. The student used "{topic}" in their code.
Here is the snippet:
```{codeSnippet}```
Explain what "{topic}" is, why it works here, and one tip to use it better.
Be concise. Max 150 words. No generic examples – reference their code directly.
```

### Example — Quiz prompt template:

```
Generate 3 short quiz questions about "{topic}" for a JS developer.
Base questions on this code context: ```{codeSnippet}```
Format: Q1: ... Answer: ... Q2: ... Answer: ... Q3: ... Answer: ...
Difficulty: beginner to intermediate. Keep answers under 2 sentences.
```

### Example — Suggest prompt template:

```
A JS/Node developer has covered these topics: {coveredList}.
Remaining topics in their curriculum: {remainingList}.
Suggest the top 3 topics to learn next. For each: topic name + 1 sentence why it builds on what
they know.
Return JSON: [{"topic":"...", "reason":"..."}, ...]
```

Store all prompt templates in data/prompts.json so you can iterate on wording without touching TypeScript.  
Load them at runtime with fs.readFileSync.

Phase 3

Streaming Groq Responses

~2 hours · makes AI feel fast and alive

Goal

Groq is fast, but streaming makes it feel instant. Instead of waiting for the full response before showing anything, you pipe each token chunk to the VS Code webview as it arrives — typewriter effect. This is the same pattern used by ChatGPT and Claude's web UI.

Steps

Enable streaming	Pass stream: true to groq.chat.completions.create(). Returns an async iterable of chunks.
Iterate chunks	for await (const chunk of stream) { const token = chunk.choices[0]?.delta?.content; postToWebview(token); }
postMessage tokens	Each token: panel.webview.postMessage({ type: 'token', content: token }). The webview appends to the DOM on each message.
Typewriter in HTML	In dashboard JS: window.addEventListener('message', e => { if(e.data.type==='token') el.textContent += e.data.content; })
Start/end signals	Send { type: 'stream_start' } before loop and { type: 'stream_end' } after. Dashboard shows spinner then hides it.
Abort controller	Allow cancel: track an AbortController per request. Wire a Stop button in the dashboard to abort the stream.

Streaming skeleton:

```
const stream = await groq.chat.completions.create({
 model: 'llama-3.3-70b-versatile', stream: true,
 messages: [{ role: 'user', content: prompt }]
});
panel.webview.postMessage({ type: 'stream_start' });
for await (const chunk of stream) {
 const tok = chunk.choices[0]?.delta?.content || '';
 panel.webview.postMessage({ type: 'token', content: tok });
}
panel.webview.postMessage({ type: 'stream_end' });
```

## Goal

Add an Explain button next to each topic in the existing TreeView. Clicking it opens a new sidebar webview (aiPanel.ts) that streams the Groq explanation. A Quiz button at the top of the panel starts quiz mode for the selected topic.

## Steps

<b>Explain command</b>	Register devmap.explainTopic command. It takes topicId as arg, pulls the code snippet from store, calls groqService.explain().
<b>Add button to tree</b>	In TopicItem, set this.command = { command:'devmap.explainTopic', arguments:[this.id] } so clicking the item triggers the command.
<b>Add inline icon</b>	Set this.contextValue = 'topic' and in package.json 'menus'.view/item/context add Explain and Quiz buttons with icons.
<b>aiPanel WebviewPanel</b>	Create a singleton WebviewPanel in VS Code's sidebar column. Re-use across explain calls — just update the content.
<b>Render AI response</b>	aiPanel HTML: dark-themed box, topic name as header, streaming text area below, copy button, quiz button.
<b>Quiz flow in sidebar</b>	Quiz button triggers devmap.quizTopic. Groq returns 3 questions. Render them one by one with a Reveal Answer toggle.

*UX tip: When the AI panel is streaming, disable the Explain/Quiz buttons to prevent overlapping requests. Re-enable on stream\_end.*

## Phase 5

# Dashboard AI Integration

~half day · full AI experience in the web panel

## Goal

Upgrade the V1 web dashboard (WebviewPanel) with three new sections: an AI Explain panel for any topic, a Quiz tab, and a Smart Suggestions card that auto-runs after every scan and shows the top 3 topics to learn next.

## Steps

<b>Topic click → explain</b>	In dashboard JS: clicking a topic pill sends <code>postMessage({ type:'explain', topicId })</code> to extension. Extension calls <code>groqService.explain()</code> and streams back.
<b>Add AI panel section</b>	New section below the progress bars: card with topic name header, streaming text box, Copy and Quiz buttons.
<b>Quiz tab</b>	Add a Quiz tab in the dashboard nav. Shows the 3 questions returned by Groq. Each has a text input + Check Answer button.
<b>Answer checking</b>	Send the user's answer + correct answer to Groq: 'Is this answer correct? Give brief feedback.' Stream the verdict.
<b>Suggestions card</b>	After every scan, extension auto-calls <code>groqService.suggest()</code> and sends result to dashboard via <code>postMessage({ type:'suggestions', data })</code> .
<b>Render suggestions</b>	Three cards in the dashboard: topic name, reason, and a Start Learning button that immediately triggers explain for that topic.
<b>Loading states</b>	Each AI section has its own spinner. Use <code>stream_start/stream_end</code> messages to toggle them independently.

## Phase 6

# Polish, Rate Limiting & Caching

~2 hours · ship-ready quality

## Goal

Groq's free tier allows 6,000 requests/day and 30 requests/min. Without caching, clicking Explain on the same topic repeatedly burns quota. Add a simple in-memory cache and a rate-limit guard.

## Steps

<b>Cache explain results</b>	Map in memory. Before calling Groq, check cache. If hit, stream cached text character-by-character for effect.
<b>Cache quiz questions</b>	Same Map. Regenerate only if user clicks Refresh Quiz.
<b>Rate limit guard</b>	Track last-call timestamp. If < 2 seconds since last call, queue the request with setTimeout. Show 'Thinking...' spinner.
<b>Suggestions cooldown</b>	Only re-run suggest() if the covered topic list has actually changed since last run. Compare with a hash of the covered set.
<b>Token cost logging</b>	Log chunk.usage?.total_tokens to VS Code's Output Channel (devmap.outputChannel) so you can see how much quota you use.
<b>User feedback</b>	After each explanation, add a thumbs up/down in the dashboard. Store locally — future use for fine-tuning prompts.



# Updated Project Structure

```
devmap/ (V2 – new files marked with ★)
■■■ src/
■ ■■■ extension.ts ← updated: new commands, groqService init
■ ■■■ scanner.ts ← unchanged from V1
■ ■■■ topicProvider.ts ← updated: Explain/Quiz buttons on items
■ ■■■ dashboardPanel.ts ← updated: AI sections, quiz tab, suggestions
■ ■■■ store.ts ← updated: stores code snippets per topic
■ ■■■ groqService.ts ★ ← explain(), quiz(), suggest() + streaming
■ ■■■ promptBuilder.ts ★ ← builds prompts from topic + code context
■ ■■■ secretStore.ts ★ ← wraps context.secrets for API key
■ ■■■ aiPanel.ts ★ ← sidebar WebviewPanel for AI responses
■ ■■■ cache.ts ★ ← in-memory cache + rate limit guard
■■■ data/
■ ■■■ topics.json ← unchanged from V1
■ ■■■ prompts.json ★ ← prompt templates for all 3 modes
■■■ media/
■ ■■■ check.svg / circle.svg ← unchanged
■ ■■■ groq-logo.svg ★ ← optional branding in dashboard
■■■ package.json ← updated: new commands, new view
■■■ tsconfig.json ← unchanged
```

# Full V2 Data Flow

Step	Actor	Action
1	File watcher	onDidSave fires → scanner.ts parses AST → detects topics + extracts code snippets
2	Store	Updates covered topics Map AND stores snippet per topicId for AI context
3	Auto-suggest	If covered set changed → groqService.suggest(coveredList, remainingList) → stream to dashboard
4	User clicks topic	TreeView command fires devmap.explainTopic(topicId)
5	promptBuilder	Loads template from prompts.json, injects topic name + code snippet
6	groqService	Calls Groq API with stream:true, iterates chunks
7	aiPanel + dashboard	Each token posted via postMessage → typewriter render in both panels
8	Cache	Full response stored in Map<topicId, string> for instant replay
9	Quiz flow	User clicks Quiz → groqService.quiz() → 3 questions rendered → answers checked via Groq

# What You Learn Building V2

Concept	Where it shows up in V2
Async iterables (for await)	Streaming Groq response chunks in groqService.ts
AbortController	Cancelling in-flight Groq stream requests
VS Code SecretStorage API	Storing the Groq API key securely

postMessage pattern	Extension ↔ WebViewPanel bidirectional messaging
Prompt engineering	Writing effective prompts in prompts.json
In-memory caching (Map)	cache.ts — avoiding redundant API calls
Rate limiting patterns	Timestamp tracking + setTimeout queue in cache.ts
JSON parsing + validation	Parsing Groq's suggest() JSON response safely
Error boundaries in Node	Wrapping every Groq call in try/catch with user-facing fallback
Template strings + injection	Building prompts dynamically in promptBuilder.ts
Singleton pattern	aiPanel.ts — one panel instance reused across calls

## V2 Setup Commands

---

### Install Groq SDK

```
npm install groq-sdk
```

### Get Groq API key

```
Sign up at console.groq.com → API Keys → Create key (free tier)
```

### Set key in DevMap

```
Cmd+Shift+P → 'DevMap: Set API Key' → paste key
```

### Recommended model

```
llama-3.3-70b-versatile (fast, smart, free tier)
```

### Test streaming locally

```
Add a test command devmap.testGroq that runs ping() and logs to Output Channel
```

### Watch + compile

```
npm run watch (same as V1)
```

### Check quota usage

```
console.groq.com → Usage dashboard – monitor daily requests
```

## V3 Ideas — After You Ship V2

---

- Switch models on the fly — let the user pick between llama3-8b (faster) and llama3-70b (smarter) in settings
  - Multi-file context — send all files where a topic appears, not just one snippet, for richer explanations
  - Spaced repetition — track which quiz questions you got wrong and resurface them after 3 days
  - Voice mode — pipe Groq text output to a TTS API and read explanations aloud while you code
  - GitHub Gist sync — save explanation history + quiz scores to a Gist so they persist across machines
  - Custom curriculum — let the user add their own topics.json entries and DevMap auto-generates AST patterns using Groq
-

